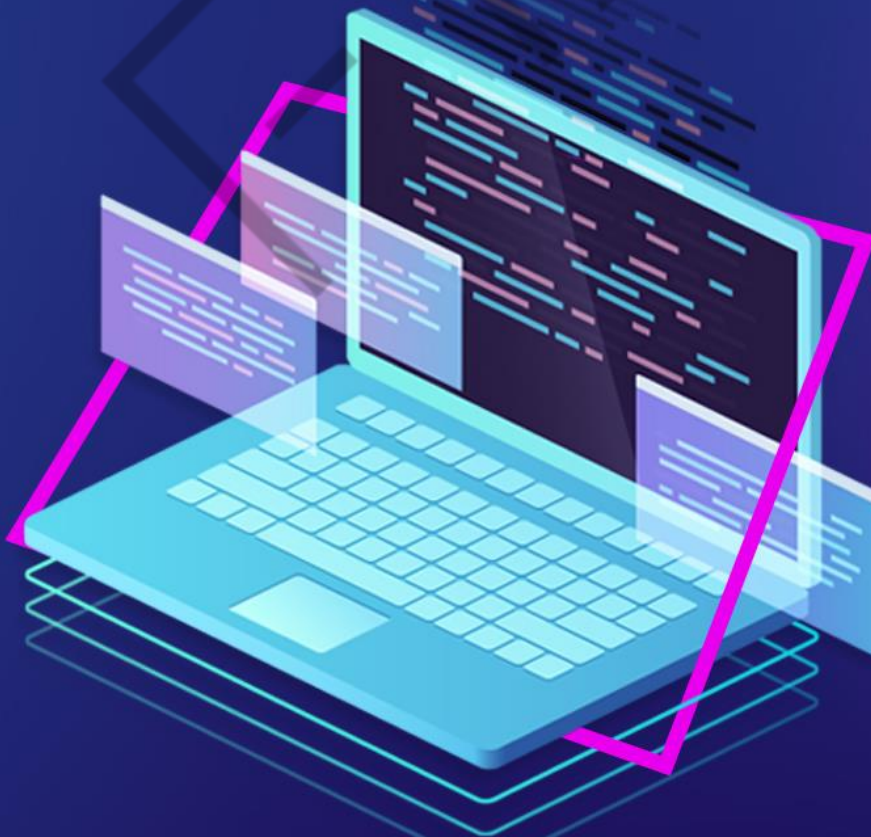


HACKER ATTACK

# *SECRETS OF THE DEVELOPERS*



5

## LISTA DE FIGURAS

|                                                            |    |
|------------------------------------------------------------|----|
| Figura 1 – <i>Hands On</i> do banco de dados MariaDB ..... | 10 |
| Figura 2 – Comandos SHOW TABLES e DESCRIBE .....           | 12 |
| Figura 3 – Resultado do comando SELECT * FROM.....         | 14 |

EXEMPLO

## LISTA DE TABELAS

|                                           |    |
|-------------------------------------------|----|
| Tabela 1 – Dados dos alunos da FIAP ..... | 10 |
|-------------------------------------------|----|

EXEMPLO

**LISTA DE CÓDIGOS-FONTE**

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Código-fonte 1 – Sintaxe SQL – Comando SELECT .....                   | 7  |
| Código-fonte 2 – Sintaxe SQL – Comando SELECT com Asterisco (*) ..... | 7  |
| Código-fonte 3 – Sintaxe SQL – Comando SELECT com WHERE .....         | 7  |
| Código-fonte 4 – Sintaxe SQL – Comando SELECT com ORDER BY .....      | 7  |
| Código-fonte 5 – Sintaxe SQL – Comando INSERT .....                   | 8  |
| Código-fonte 6 – Sintaxe SQL – Comando UPDATE .....                   | 9  |
| Código-fonte 7 – Sintaxe SQL – Comando DELETE .....                   | 9  |
| Código-fonte 8 – Comando CREATE DATABASE .....                        | 11 |
| Código-fonte 9 – Comando CREATE TABLE .....                           | 11 |
| Código-fonte 10 – Comandos SHOW TABLES e DESCRIBE .....               | 11 |

## SUMÁRIO

|                                                                   |    |
|-------------------------------------------------------------------|----|
| ESCOPO DE VARIÁVEL E VULNERABILIDADE .....                        | 6  |
| 1 CONSIDERAÇÕES INICIAIS .....                                    | 6  |
| 2 COMANDOS DML E CRUD .....                                       | 6  |
| 2.1 O comando SELECT .....                                        | 7  |
| 2.2 O comando INSERT .....                                        | 8  |
| 2.3 O comando UPDATE .....                                        | 8  |
| 2.4 O comando DELETE .....                                        | 9  |
| 3 PARTINDO PARA O ESTUDO DE CASO .....                            | 9  |
| 3.1 <i>Hands On</i> – Estudo de Caso aplicado com DDL e DML ..... | 9  |
| 3.2 Estrutura da tabela de dados .....                            | 10 |
| 3.3 Montagem dos comandos DDL .....                               | 10 |
| 3.4 Montagem dos comandos DML .....                               | 13 |
| REFERÊNCIAS .....                                                 | 15 |

## ESCOPO DE VARIÁVEL E VULNERABILIDADE

### 1 CONSIDERAÇÕES INICIAIS

Pudemos ter uma breve ideia dos principais comandos DDL (*Data Definition Language*). Mas precisaremos nos aprofundar um pouco mais neste assunto. O objetivo é preparar melhor o terreno para o desenvolvimento dos próximos temas.

Precisamos, agora, sobretudo, entender a sintaxe dos comandos DML (*Data Manipulation Language*), a dinâmica de funcionamento das quatro operações de um CRUD (*Create, Read, Update e Delete*) e, por fim, a modelagem e o gerenciamento de um banco de dados de forma prática (*hands on*).

Para tal, faremos uma breve abordagem das operações de CRUD e seu funcionamento com a sintaxe simplificada para facilitar a consulta. Finalizaremos com um caso prático simples que ajudará a internalizar este conteúdo fundamental para que possamos entender melhor o funcionamento dos bancos de dados e suas vulnerabilidades. Vamos lá?!

### 2 COMANDOS DML E CRUD

Como já mencionamos, o DML (*Data Manipulation Language*) é o responsável pela manipulação das tabelas de um banco de dados. Uma linguagem utilizada para inclusão, exclusão, recuperação e atualização das informações de um banco de dados. Esse conjunto de instruções é bem intuitivo e composto por alguns comandos:

**SELECT:** recupera as informações no banco de dados.

**INSERT:** adiciona novas informações no banco de dados.

**UPDATE:** altera informações no banco de dados.

**DELETE:** deleta informações no banco de dados.

## 2.1 O comando SELECT

O comando **SELECT** lista, consulta e obtém dados de uma tabela. A sintaxe do comando SELECT é a seguinte:

```
SELECT [(<nome-coluna>, [<nome-coluna>], ...)]  
FROM <nome-tabela>;
```

Código-fonte 1 – Sintaxe SQL – Comando SELECT  
Fonte: adaptado de Price (2009)

Essa obtenção de dados pode ser feita utilizando alguns parâmetros, como nome da tabela ou da coluna, realizar contagem (**COUNT**), agrupamento (**GROUP BY**), filtragem (**WHERE**) ou mudança de ordem dos resultados (**ORDER BY**) e cruzamento entre tabelas (**JOIN**). O asterisco (\*) pode ser usado quando a informação que se quer obter se refere a todas as colunas, com a seguinte sintaxe:

```
SELECT *  
FROM <nome-tabela>;
```

Código-fonte 2 – Sintaxe SQL – Comando SELECT com Asterisco (\*)  
Fonte: adaptado de Price (2009)

Na sintaxe acima, todas as colunas serão selecionadas.

Podemos, ainda, utilizar o termo **WHERE** no final do comando SELECT, para obter um conjunto de registros específicos de uma tabela, com a seguinte sintaxe:

```
SELECT [(<nome-coluna>, [<nome-coluna>], ...)]  
FROM <nome-tabela>  
WHERE <condições>;
```

Código-fonte 3 – Sintaxe SQL – Comando SELECT com WHERE  
Fonte: adaptado de Price (2009)

E para que os resultados sejam listados ordenados, usamos o **ORDER BY**, com a sintaxe:

```
SELECT [(<nome-coluna>, [<nome-coluna>], ...)]  
FROM <nome-tabela>  
ORDER BY (<nome-coluna>, [<nome-coluna>], ...)  
(ASC | DESC) ;
```

Código-fonte 4 – Sintaxe SQL – Comando SELECT com ORDER BY

Fonte: adaptado de Price (2009)

Utilizando, no caso da sintaxe acima descrita, o '**ASC**' e o '**DESC**' ao final do comando (após o ORDER BY) para a ordenação ascendente ou descendente, respectivamente.

## 2.2 O comando INSERT

O comando **INSERT** insere, cadastra e inclui registros em uma tabela, um registro por vez, com a seguinte sintaxe básica:

```
INSERT INTO <nome-tabela>
    [(<nome-coluna>, [<nome-coluna>], ...)]
VALUES
    (<conteúdo>, [<conteúdo>], ...);
```

Código-fonte 5 – Sintaxe SQL – Comando INSERT  
Fonte: adaptado de Price (2009)

**1ª Observação:** caso os dados a serem inseridos estejam na mesma ordem em que as colunas foram criadas, os nomes das colunas (comando acima) podem ser omitidos.

**2ª Observação:** ainda no comando acima, somente após o comando **VALUES**, os dados serão inseridos.

## 2.3 O comando UPDATE

O comando **UPDATE** atualiza, altera e modifica dados de uma tabela. Ao contrário do comando anterior, o UPDATE pode atualizar mais de uma coluna simultaneamente, ou seja: com apenas um UPDATE, é possível atualizar até a tabela inteira.

A sintaxe básica do comando UPDATE é a seguinte:

```
UPDATE <nome-tabela>
SET <nome-coluna> = <novo conteúdo para o campo>
    [, <nome-coluna> = <novo conteúdo para o campo>]
[WHERE <condição>];
```



Código-fonte 6 – Sintaxe SQL – Comando UPDATE  
Fonte: adaptado de Price (2009)

Os novos valores para os campos a serem alterados devem ser colocados após o comando **SET**. No caso de não ser preciso atualizar os campos da tabela inteira, utiliza-se o **WHERE**, após o qual inserimos as condições específicas.

## 2.4 O comando DELETE

Por fim, o comando **DELETE** deleta, exclui e elimina dados de uma tabela. Como o comando anterior, pode remover mais de uma coluna simultaneamente, ou seja: com apenas um DELETE, é possível remover até a tabela inteira. Sua sintaxe básica:

```
DELETE FROM <nome-tabela>  
[WHERE <condição>];
```

Código-fonte 7 – Sintaxe SQL – Comando DELETE  
Fonte: adaptado de Price (2009)

A cláusula **WHERE** é opcional, porém é indicado seu uso para que apenas os registros corretos sejam removidos, caso contrário, todos os registros da tabela serão removidos.

## 3 PARTINDO PARA O ESTUDO DE CASO

### 3.1 Hands On – Estudo de Caso aplicado com DDL e DML

Agora que vimos os comandos DDL e DML, podemos partir para o estudo de caso. Nele, podemos observar, de forma prática, a modelagem e a manipulação de um banco de dados utilizando MariaDB. Lembre-se de que os comandos SQL utilizados são amplamente aceitos em outros SGBDs, e que o MariaDB é um projeto bem maduro e consolidado, amplamente usado e universal, muito disso graças ao seu propósito de ser uma continuidade Open Source do já aclamado MySQL. Então, mãos à obra!



Figura 1 – Hands On do banco de dados MariaDB  
Fonte: Elaborado pelo autor (2020)

### 3.2 Estrutura da tabela de dados

A estratégia que utilizaremos para resolver este estudo de caso seguirá as duas etapas básicas de modelagem de um banco de dados: o uso dos comandos DDL para criação e verificação das tabelas com as colunas segundo o caso exposto a seguir; e a utilização dos comandos DML para manipulação desses registros, com a inserção dos dados e posterior verificação.

Adotaremos a seguinte problemática como caso de estudo: recebemos a tarefa de criar uma tabela no banco de dados que deverá conter os dados cadastrais dos alunos da FIAP, conforme os campos e dados a seguir. Após isso, deveremos manipular esse banco de dados, utilizando, para isso, os comandos aprendidos até aqui.

| FIAP   |                 |            |       |
|--------|-----------------|------------|-------|
| Alunos |                 |            |       |
| Código | Nome            | Identidade | Idade |
| 1      | Carlos Fernando | 032415299  | 22    |
| 2      | José Silva      | 154236768  | 23    |
| 3      | Robson Franco   | 985634217  | 25    |

Tabela 1 – Dados dos alunos da FIAP  
Fonte: Elaborado pelo autor (2020)

### 3.3 Montagem dos comandos DDL

Iniciaremos este *hands on* utilizando o comando **CREATE DATABASE** e o comando **USE**, conforme visto anteriormente:

```
CREATE DATABASE fiap;
```

```
USE fiap;
```

Código-fonte 8 – Comando CREATE DATABASE  
Fonte: Elaborado pelo autor (2020)

Em seguida, usaremos apenas um comando **CREATE TABLE**, visto que temos apenas uma tabela a ser criada:

```
CREATE TABLE ALUNOS (  
  CODIGO INT(4) AUTO_INCREMENT,  
  NOME VARCHAR(200) NOT NULL,  
  IDENTIDADE VARCHAR(20) NOT NULL,  
  IDADE INT(2) NOT NULL,  
  PRIMARY KEY (CODIGO)  
);
```

Código-fonte 9 – Comando CREATE TABLE  
Fonte: Elaborado pelo autor (2020)

Note que a especificação das colunas possui a seguinte ordem: <nome-coluna> <tipo-do-dado>, com vírgulas entre cada especificação de coluna. O tipo de dado leva em consideração a quantidade de caracteres que serão admitidos (entre parênteses).

Podemos, agora, visualizar o resultado do comando acima utilizando os seguintes comandos:

```
SHOW TABLES;
```

```
DESCRIBE ALUNOS;
```

Código-fonte 10 – Comandos SHOW TABLES e DESCRIBE  
Fonte: Elaborado pelo autor (2020)

O resultado será o seguinte:

```

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| fiap     |
| information_schema |
| mysql    |
| performance_schema |
+-----+
4 rows in set (0.001 sec)

MariaDB [(none)]> USE fiap;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [fiap]> SHOW TABLES;
+-----+
| Tables_in_fiap |
+-----+
| ALUNOS          |
+-----+
1 row in set (0.001 sec)

MariaDB [fiap]> DESCRIBE ALUNOS;
+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+
| CODIGO     | int(4)        | NO   | PRI | NULL    | auto_increment |
| NOME       | varchar(200)  | NO   |     | NULL    |                |
| IDENTIDADE | varchar(20)    | NO   |     | NULL    |                |
| IDADE      | int(2)        | NO   |     | NULL    |                |
+-----+
4 rows in set (0.002 sec)

MariaDB [fiap]>

```

Figura 2 – Comandos SHOW TABLES e DESCRIBE  
Fonte: Elaborado pelo autor (2020)

**Observação:** observe que, com o comando **SHOW DATABASES** utilizado no exemplo mencionado (Figura “Comandos SHOW TABLES e DESCRIBE”), é possível verificar que a base de dados "fiap" foi criada com sucesso.

Ao utilizar o comando **DESCRIBE ALUNOS**, são mostrados os campos da tabela, o tipo de dado, Chave Primária, Autoincrement e Not-null. A partir daí, podemos popular nossa tabela com os dados dos alunos da FIAP.

Importante:

**NOT NULL:** este atributo de campo evita erros causados por um valor nulo inserido erroneamente (campo de preenchimento obrigatório).

**AUTO\_INCREMENT:** instrui o MariaDB a adicionar o próximo valor disponível ao campo CODIGO, para não haver repetições na mesma tabela.

**PRIMARY KEY:** define uma coluna como Chave Primária (no exemplo dado, a coluna CODIGO).

### 3.4 Montagem dos comandos DML

Agora, iremos usar o comando **INSERT** para inserir os dados referentes a três alunos da FIAP, conforme os dados a seguir:

```
INSERT INTO ALUNOS
(CODIGO, NOME, IDENTIDADE, IDADE)
VALUES
(null, 'Carlos Fernando', '032415299', '22');

INSERT INTO ALUNOS
(CODIGO, NOME, IDENTIDADE, IDADE)
VALUES
(null, 'José Silva', '154236768', '23');

INSERT INTO ALUNOS
(CODIGO, NOME, IDENTIDADE, IDADE)
VALUES
(null, 'Robson Franco', '985634217', '25');
```

Código-fonte 11 – Comandos INSERT para inserção de dados  
Fonte: Elaborado pelo autor (2020)

**Observação:** quando é inserido texto ou uma string, o valor deve ser colocado entre aspas simples (').

O console responde aos comandos com as linhas (rows) inseridas corretamente; podemos, ainda, verificar se a inserção dos dados foi feita de maneira correta por meio de um comando **SELECT**:

```
SELECT * FROM ALUNOS;
```

Código-fonte 12 – Exemplo de comando SELECT  
Fonte: Elaborado pelo autor (2020)

O resultado da operação aparecerá no console, conforme a Figura “Resultado do comando SELECT \* FROM”, listando os dados dos três alunos inseridos:

```
root@fiap:~# mariadb
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 36
Server version: 10.3.25-MariaDB-0+deb10u1 Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use fiap;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [fiap]> INSERT INTO ALUNOS (CODIGO, NOME, IDENTIDADE, IDADE) VALUES (null, 'Carlos Fernando', '032415299', '22');
Query OK, 1 row affected (0.029 sec)

MariaDB [fiap]> INSERT INTO ALUNOS (CODIGO, NOME, IDENTIDADE, IDADE) VALUES (null, 'José Silva', '154236768', '23');
Query OK, 1 row affected (0.044 sec)

MariaDB [fiap]> INSERT INTO ALUNOS (CODIGO, NOME, IDENTIDADE, IDADE) VALUES (null, 'Robson Franco', '985634217', '25');
Query OK, 1 row affected (0.049 sec)

MariaDB [fiap]> SELECT * FROM ALUNOS;
+-----+-----+-----+-----+
| CODIGO | NOME      | IDENTIDADE | IDADE |
+-----+-----+-----+-----+
| 1      | Carlos Fernando | 032415299 | 22    |
| 2      | José Silva    | 154236768 | 23    |
| 3      | Robson Franco  | 985634217 | 25    |
+-----+-----+-----+-----+
3 rows in set (0.001 sec)

MariaDB [fiap]> _
```

Figura 3 – Resultado do comando SELECT \* FROM

Fonte: Elaborado pelo autor (2020)

A partir daqui, as possibilidades são imensas. Para mais informações e possibilidades que o banco de dados MariaDB oferece, aconselha-se a leitura da sua documentação, que pode ser acessada no site oficial do MariaDB:

<<https://mariadb.com/kb/en/documentation/>>

O conhecimento dessa estrutura de banco de dados que acabamos de estudar é fundamental para avaliarmos suas vulnerabilidades e como blindá-la contra ataques maliciosos. Cabe ressaltar a grandiosidade do assunto, sendo necessário, portanto, aprofundar-se no tema conforme as peculiaridades de cada projeto.

## REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados: fundamentos e aplicações**. 6. ed. São Paulo: Pearson Addison-Wesley, 2011.

MACHADO, F. N. R. **Banco de Dados – Projeto e Implementação**. 2. ed. São Paulo: Érica, 2008.

MARIADB. **MariaDB Documentation: MariaDB Basics**. 2020. Disponível em: <<https://mariadb.com/kb/en/mariadb-basics/>>. Acesso em: 29 nov. 2020.

PRACIANO, E. **Tipos de dados do MySQL e MariaDB**. 2014. Disponível em: <<https://elias.praciano.com/2014/01/mysql-tipos-de-dados/>>. Acesso em: 29 nov. 2020.

PRICE, J. **Database 11g SQL Domine SQL e PL/SQL no banco de dados Oracle**. Porto Alegre: Bookman, 2009.

QUESADA, A. *et al.* **Apostila de MySQL**. Coordenadoria do Governo Eletrônico (CGE) – Equipe de Treinamento Técnico, 2001. Disponível em: <[http://www.telecentros.sp.gov.br/saber/apostilas/antigas/apostila\\_sql.pdf](http://www.telecentros.sp.gov.br/saber/apostilas/antigas/apostila_sql.pdf)>. Acesso em: 29 nov. 2020.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.